GUIA TÉCNICO | 11

Regularização, BatchNorm, Dropout e callbacks

Capítulo 11/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Identificar e combater overfitting com métodos apropriados.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

L2 penaliza magnitude de pesos, dropout desativa unidades aleatoriamente durante treinamento, BatchNorm normaliza ativações por lote; EarlyStopping monitora validação. Cuidado com conjuntos de validação pequenos e parada prematura.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: $L_{total}=L_{dados}+\lambda\ ||W||^2$

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Regularização de pesos, dropout e early stopping têm mecanismos distintos. A validação para interrupção deve ficar dentro do treino.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
rng=np.random.default_rng(12)
W=np.array([1.,-2.,.5]);lam=.1
pen=lam*np.sum(W**2)
mask=(rng.random(10000)>.2).astype(int)/.8
print('L2=',pen,'dropout média do multiplicador=',round(mask.mean(),3))
assert abs(mask.mean()-1)<.06
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um

conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Regularização e early stopping em treinamento controlado

Regularização de pesos, dropout e early stopping têm mecanismos distintos. A validação para interrupção deve ficar dentro do treino.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import numpy as np
X,y=make_moons(n_samples=300,noise=.25,random_state=13)
Xtr,Xte,ytr,yte=train_test_split(X,y,stratify=y,test_size=.3,random_state=13)
m=MLPClassifier(hidden_layer_sizes=(32,16),alpha=.05,early_stopping=True,validation_fracti  ↪
    on=.2,n_iter_no_change=12,max_iter=400,random_state=13)
p=make_pipeline(StandardScaler(),m).fit(Xtr,ytr)
print('Épocas efetivas:',m.n_iter_,'melhor escore validação:',m.best_validation_score_)
print('Acurácia teste:',round(p.score(Xte,yte),3));assert m.n_iter_<=400
```

O early stopping envolve uma divisão de validação dentro do treino; não utilizar o teste para decidir quando parar.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que o Dropout é normalmente desativado na inferência?

Resposta: Porque o modelo deve produzir previsões estáveis usando a rede completa; a escala é ajustada no treinamento.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.

• Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide